

UNIT – 1
ASSESSMENT – 2
SOLUTIONS

NIHARIKA PREEJESH
192524506

1. i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the node with the smallest heuristic value ($h(n)$), meaning it always moves toward the estimated closest path to the goal.

Formula:

$$f(n) = h(n)$$

Advantages

- Fast decision-making
- Less memory usage
- Suitable for urgent situations

Disadvantages

- Ignores actual travel cost
- May choose unsafe or blocked routes
- Does not always find the optimal path

ii. A Search*

A* Search considers both the actual cost travelled and the estimated distance to the goal.

Formula:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = Actual cost
- $h(n)$ = Estimated cost to goal

Advantages

- Finds the optimal path
- Considers terrain and weather costs
- More reliable in dynamic environments

iii. Impact of Heuristic Accuracy

Greedy BFS

Highly dependent on heuristic accuracy

Poor heuristic may lead to wrong routes

A more accurate heuristic improves the performance of both algorithms.

A* Search

Uses both heuristic and actual cost

Still performs well even with moderate heuristic errors

iv. Effect of Dynamic Changes

When roads become blocked:

- **GBFS** may continue toward blocked paths and require restarting the search.
- **A*** recalculates the path by considering updated costs, making it more adaptable.

v. Recommended Algorithm

Recommended: A*

Justification

- Finds the shortest and safest route
- Handles varying travel costs effectively
- Adapts better to changing environments
- Provides more reliable navigation during emergencies

2. Problem Formulation

The objective is to find the best traffic signal timings that minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

This is an **optimization problem** because the AI searches for the best possible solution among many signal timing combinations.

Why Traditional Search is Inefficient

- Huge number of possible signal combinations.
- Traffic conditions change continuously.
- Searching every possible solution takes too much time.
- Not suitable for real-time traffic management.

i. Hill Climbing

Hill Climbing starts with an initial solution and continuously moves to a neighboring solution if it improves traffic flow.

Advantages

- Simple to implement.
- Fast execution.
- Suitable for small optimization problems.

Limitations

- May stop at a local optimum.
- Cannot always find the best global solution.
- Sensitive to the starting solution.

ii. Local Maxima, Plateaus, and Ridges

- **Local Maxima:** The algorithm finds a good solution but not the best one.
 - *Example:* A traffic signal pattern improves one intersection but increases congestion elsewhere.

- **Plateaus:** Many neighboring solutions have the same value, so the algorithm cannot decide where to move.
 - *Example:* Different signal timings produce nearly identical traffic flow.
- **Ridges:** The best solution requires moving in a direction that simple neighboring steps cannot easily reach.
 - *Example:* Improving traffic requires changing multiple intersections simultaneously.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

- Occasionally accepts worse solutions.
- Helps escape local maxima.
- Explores more possible solutions.

Genetic Algorithm

- Uses selection, crossover, and mutation.
- Evaluates multiple solutions simultaneously.
- Produces better solutions for large and complex problems.

iv. Recommended Approach

Recommended: Genetic Algorithm

Justification

- Handles large search spaces efficiently.
- Adapts to changing traffic conditions.
- Finds high-quality solutions.
- Reduces congestion, waiting time, and fuel consumption.

3. i. Why Online Search is Required

An **online search agent** makes decisions while exploring the environment because the complete map is not known in advance.

Unlike offline search, the rover learns about obstacles and paths only after moving through the terrain.

Advantages

- Works without a complete map.
- Adapts to newly discovered obstacles.
- Suitable for real-time exploration.

ii. Challenges of Partial Observability and Dynamic Environments

Partial Observability

- The rover cannot see the entire terrain.
- Hazards are detected only when they come within sensor range.

Dynamic Environment

- Terrain conditions may change unexpectedly.
- Communication delays prevent constant guidance from Earth.

These challenges require the rover to make safe decisions independently.

iii. Internal Model Update

The rover continuously updates its internal map by:

- Collecting sensor data.
- Identifying safe and unsafe paths.
- Recording explored locations.
- Planning new routes based on updated information.

This allows the rover to improve navigation as it explores.

iv. Comparison of Search Strategies

Online Search	Uninformed Search	Informed Search
Learns while moving	No heuristic used	Uses heuristic information
Suitable for unknown environments	Requires complete search	Works better when map is known

Online Search	Uninformed Search	Informed Search
Adapts to changes	Less efficient	Faster but assumes better knowledge

v. Recommended Approach

Recommended: Online Search Agent

Justification

- Handles unknown environments effectively.
- Updates decisions using real-time sensor data.
- Improves safety by avoiding hazards.
- Adapts as new information becomes available.

4. i. Constraint Satisfaction Problem (CSP)

The exam scheduling problem is modeled as a **Constraint Satisfaction Problem (CSP)** because it requires assigning exam slots while satisfying several constraints.

Variables

- Exam subjects

Domains

- Available dates
- Time slots

- Exam halls

Constraints

- No student has overlapping exams.
- Exam hall capacity should not exceed the limit.
- Faculty must be available.
- Some subjects must be scheduled before others.
- Special accommodation students receive appropriate seating.

ii. Backtracking Search

Backtracking Search assigns one exam at a time and checks whether all constraints are satisfied.

If a conflict occurs, it returns to the previous assignment and tries another option.

Advantages

- Finds a valid timetable.
- Simple to implement.
- Guarantees a solution if one exists.

Limitation

- Becomes slow as the number of exams increases.

iii. Forward Checking

Forward Checking improves Backtracking by removing invalid choices before assigning the next variable.

Benefits

- Detects conflicts early.
- Reduces unnecessary searching.
- Improves execution speed.
- Suitable for large scheduling problems.

iv. Real-World Challenges

- Large number of students and courses.
- Last-minute faculty changes.
- Limited exam halls.
- Special accommodation requirements.
- Timetable modifications due to unforeseen events.

v. Recommended Strategy

Recommended: Backtracking Search with Forward Checking

Justification

- Produces valid exam schedules.
- Reduces search time.
- Handles multiple constraints efficiently.

- Scales better for large universities.

5. i. Minimax Algorithm

The **Minimax algorithm** is used in two-player games where one player tries to maximize the score while the opponent tries to minimize it.

The AI evaluates possible future moves and chooses the one that gives the best outcome assuming the opponent also plays optimally.

Advantages

- Makes intelligent decisions.
- Finds the best possible move.
- Suitable for strategic games.

ii. Computational Challenges

Large game trees create several challenges:

- Huge number of possible moves.
- High memory usage.
- Longer decision-making time.
- Difficult to search every possible game state in real time.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by ignoring branches that cannot affect the final decision.

Benefits

- Reduces unnecessary calculations.
- Speeds up decision-making.
- Produces the same optimal result as Minimax.
- Makes real-time game AI more efficient.

iv. Evaluation Function

The evaluation function assigns a score to each game state.

Factors Considered

- Player's score.
- Number of available moves.
- Resources collected.
- Health of units.
- Control of important locations.

A higher score indicates a stronger game position.

v. Recommended Strategy

Recommended: Minimax with Alpha-Beta Pruning

Justification

- Produces optimal decisions.

- Reduces computation time.
- Handles large game trees efficiently.
- Suitable for real-time strategy games.